



EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 8

Object-Oriented Analysis — Behavioural Modelling

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 8 of 16



Contents

1. Week 8 Introduction	3
2. Learning Outcomes	3
PART A — WHAT IS BEHAVIOURAL MODELLING?	3
3. Definition and Why We Need It	3
PART B — USE CASES	4
4. What Is a Use Case?	4
5. Actors	5
PART C — USE CASE DIAGRAMS	6
6. The Use Case Diagram and System Boundary	6
7. Use Case Relationships — include, extend, generalisation	6
PART E — USE CASE SPECIFICATIONS	8
8. The Diagram Is Not Enough	8
PART F & G — INTERACTION AND SEQUENCE DIAGRAMS	8
9. Sequence Diagrams	8
10. Alternative Sequences — alt, loop, opt	10
PART I — COMMUNICATION (COLLABORATION) DIAGRAMS	10
11. Communication Diagrams	10
PART J — ACTIVITY DIAGRAMS	11
12. Activity Diagrams	11
13. Swimlanes	13
PART M — HOW THE MODELS WORK TOGETHER	13
14. Use Case □ Activity □ Sequence	13
15. Full Analysis Documentation	14
16. Common Student Confusions	15
17. Practical Lab and Tutorial	15
18. Examination-Style Questions	15
19. Week 8 Summary and Key Terms	16
20. References and Further Reading	16

1. Week 8 Introduction

The progression matters: Weeks 4–6 modelled **what things exist** (object modelling); Week 7 modelled **how objects change state over time** (dynamic modelling); Week 8 models **how users and objects interact to accomplish tasks** (behavioural modelling). Week 9 turns to the development process itself, and Weeks 10+ to design.

Behavioural modelling is not simply another collection of UML diagrams. The major idea is: **behavioural modelling describes what the system does, how users interact with it, how objects communicate, and how activities flow to achieve a goal**. The official outline covers use cases, use case diagrams, interaction diagrams, sequence diagrams, collaboration (communication) diagrams, activity diagrams and full analysis documentation — all seven are covered this week.

2. Learning Outcomes

By the end of this week, a student should be able to:

- Define behavioural modelling and explain why it is important.
- Define use cases and actors, and construct a use case diagram.
- Explain the system boundary and include, extend and generalization relationships.
- Write a detailed use-case specification.
- Construct sequence diagrams with lifelines, activation bars and messages.
- Explain synchronous/asynchronous messages, and alt/loop/opt fragments.
- Construct communication (collaboration) diagrams.
- Explain the difference between sequence and communication diagrams.
- Construct activity diagrams with decisions, parallel flows and swimlanes.
- Connect use cases, activity diagrams and interaction diagrams into coherent analysis documentation.

PART A — WHAT IS BEHAVIOURAL MODELLING?

3. Definition and Why We Need It

Behavioural modelling is the process of representing how a system behaves in response to users, events, inputs and other objects — in simple language, it tells us **what the system does and how things interact while it is doing it**. “Build a university registration system” is far too vague; we need to know who uses it, what each role can do, what happens when registration succeeds or fails, what messages pass between objects, and which activities happen in parallel. Behavioural models answer these questions.

Think of behavioural modelling as a **family**: use cases (use case diagrams), interactions (sequence and communication diagrams) and activities (activity diagrams). Each diagram answers a slightly different question, summarised below — this is very important for examinations.

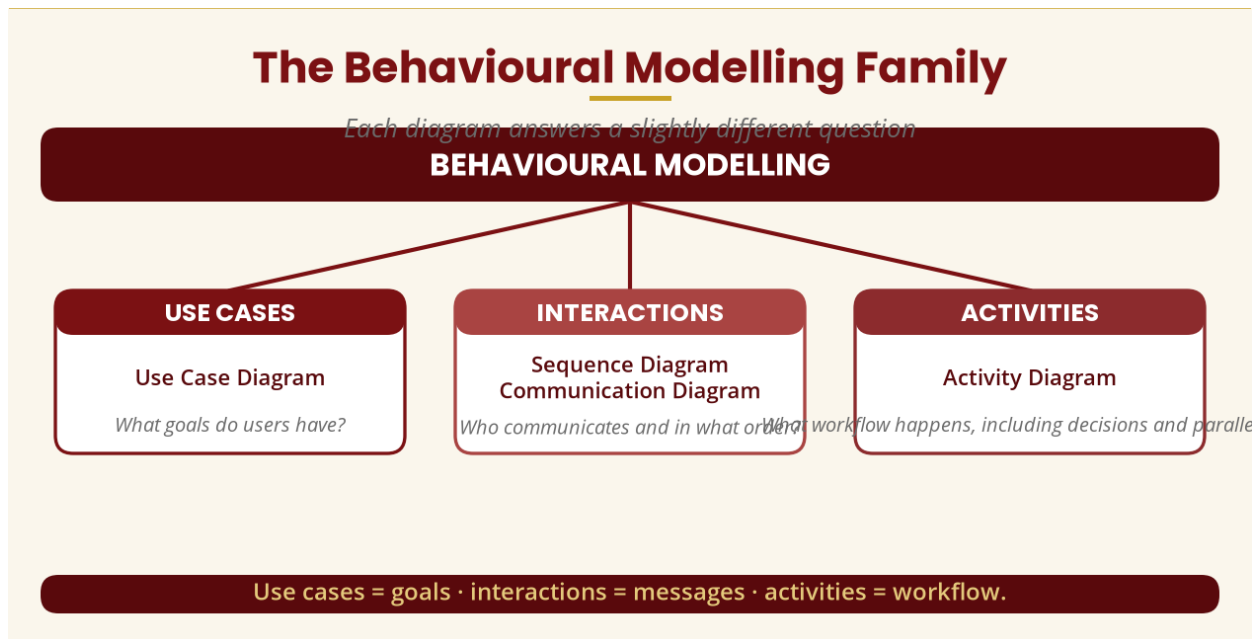


Figure 1 — The behavioural modelling family

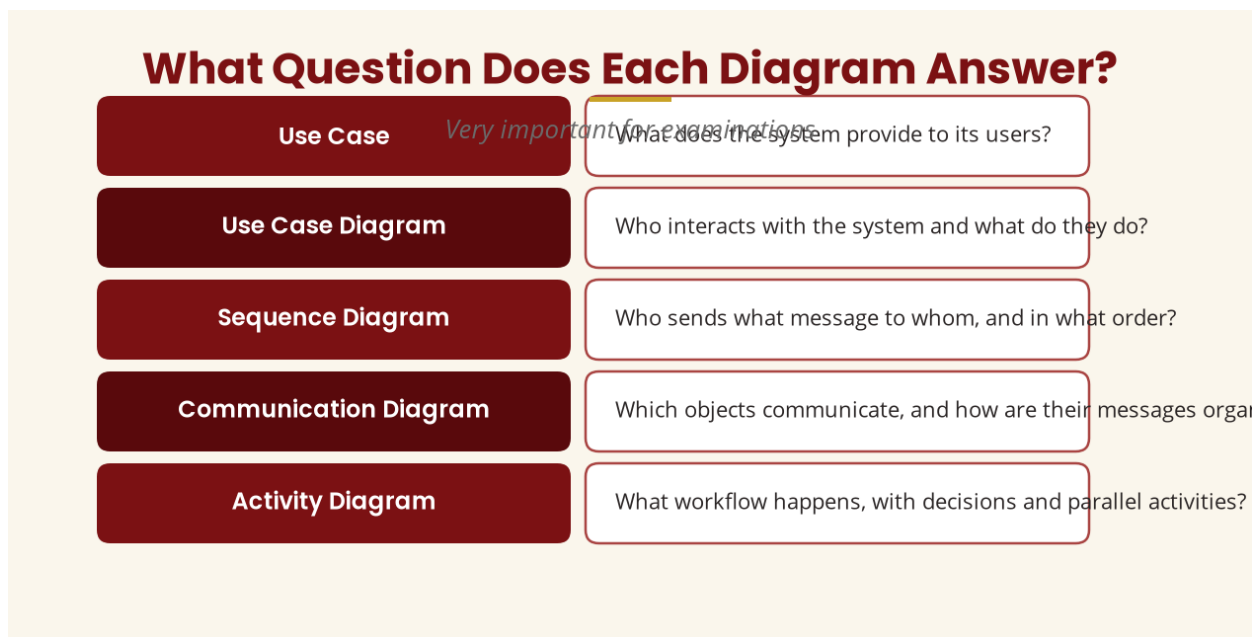


Figure 2 — What question each diagram answers

PART B — USE CASES

4. What Is a Use Case?

A **use case** describes a goal-oriented interaction between an actor and a system that produces a result of value to the actor — something a user wants to accomplish. Examples: Login, Register for Course, Make Payment, Withdraw Money, Place Order, Track Order, Generate Report, Submit Assignment, Book Appointment.

When identifying a use case, ask: **“What does the actor want to accomplish?”** A student saying “I want to register for a course” gives the use case **Register for Course** — not *Click*

Register Button, which is only one interface action. A use case is the user's goal; the functions/actions (read card, validate PIN, check balance, dispense cash) are the steps used to achieve it.

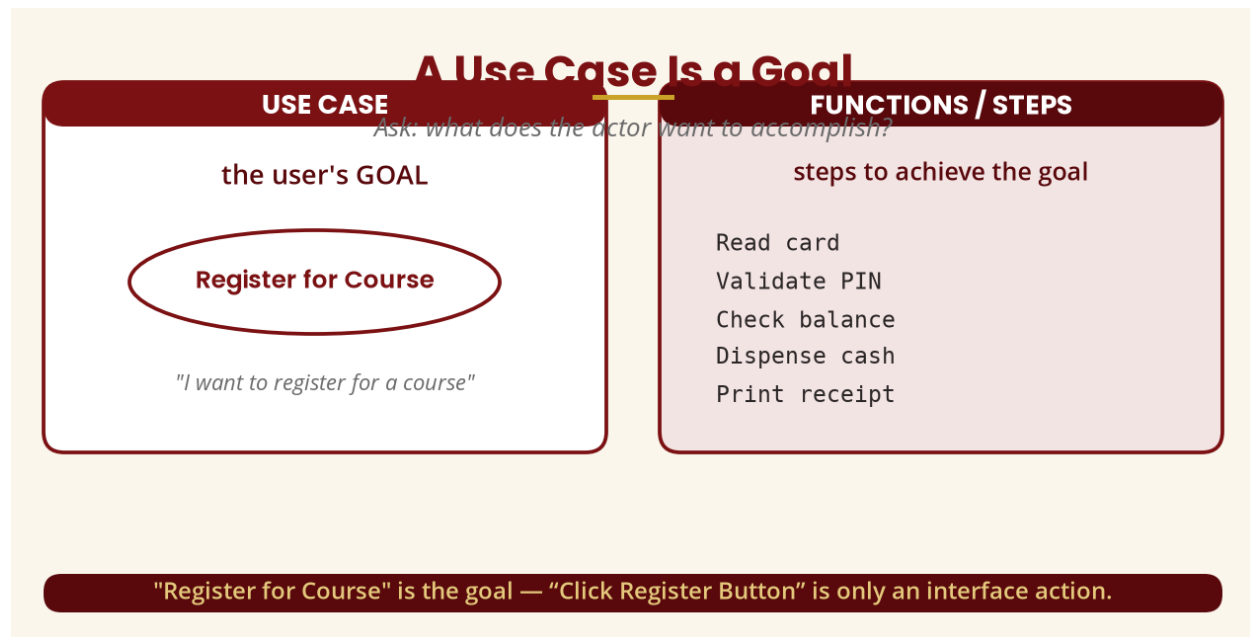


Figure 3 — A use case is a goal, not a function

5. Actors

An **actor** is an external role that interacts with the system, outside the system boundary — Student, Lecturer, Administrator, Customer, Bank, Payment Gateway, Email Service. An actor represents a **role**: one person who is both a Student and a Teaching Assistant plays two roles. An actor does not have to be human — a Payment Gateway that is external to the system being modelled is also an actor.

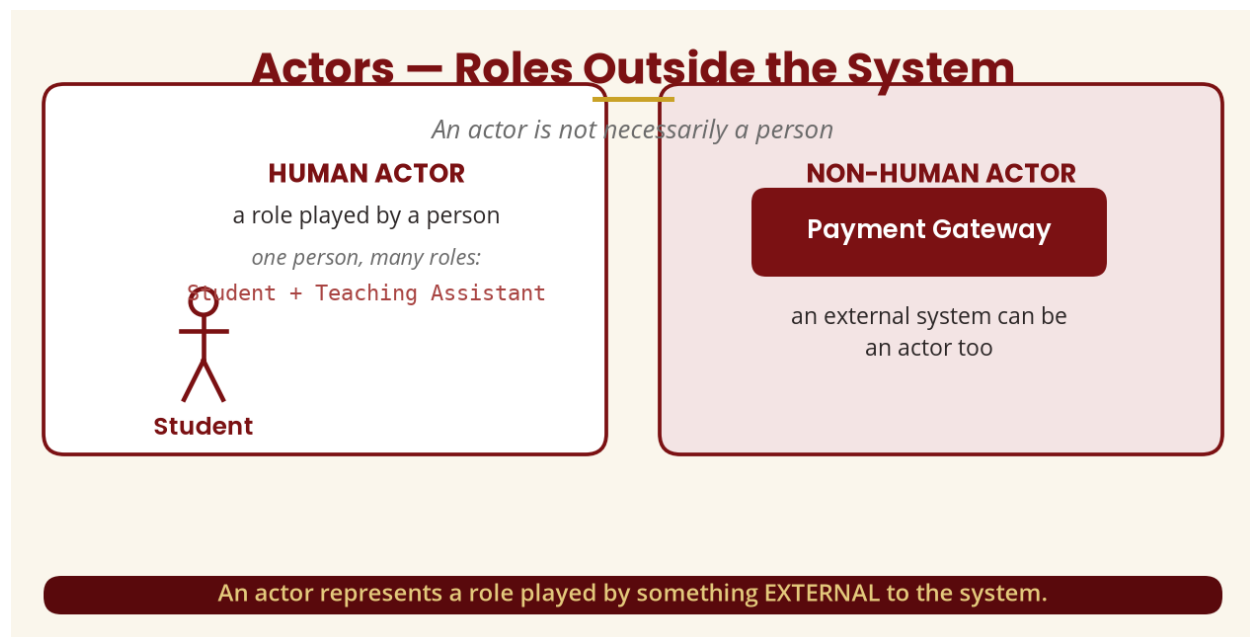


Figure 4 — Human and non-human actors

PART C — USE CASE DIAGRAMS

6. The Use Case Diagram and System Boundary

A **use case diagram** provides a high-level view of actors, use cases, the system boundary and their relationships — it answers “**who interacts with the system and what goals can they accomplish?**” The **system boundary** is drawn as a rectangle containing the use cases; actors sit outside it. A basic association line shows that an actor participates in a use case: Student — (Register Course).

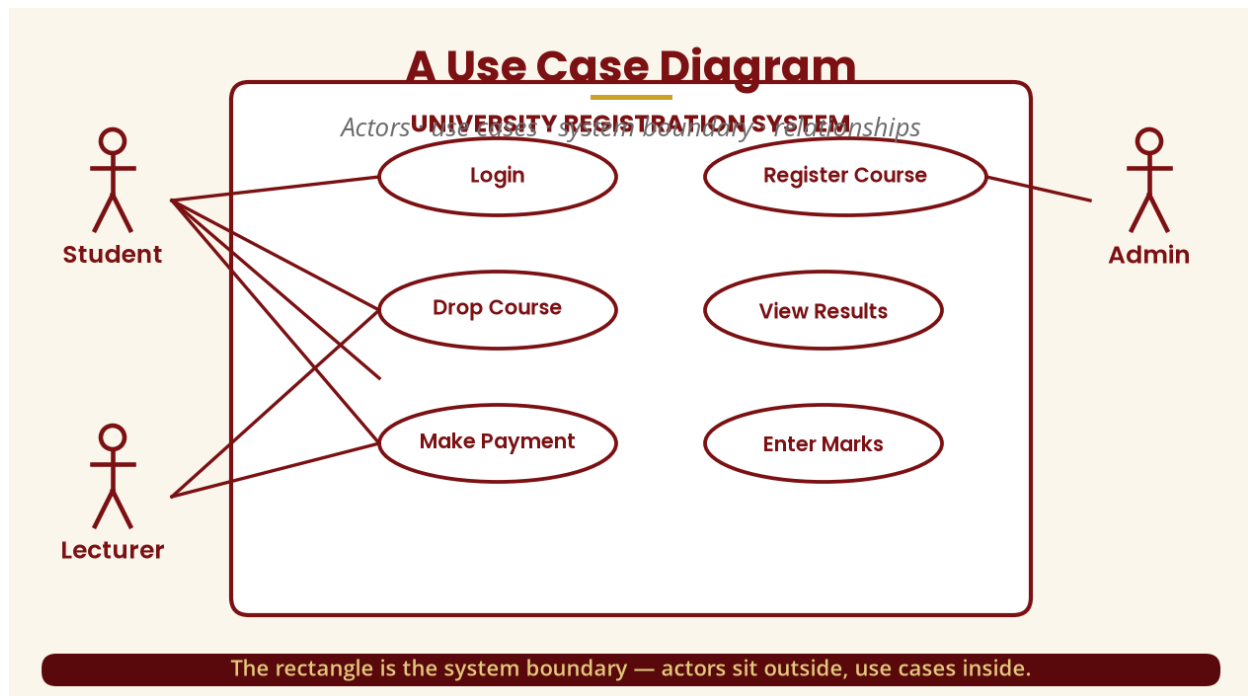


Figure 5 — A complete use case diagram

7. Use Case Relationships — include, extend, generalisation

Large systems can have dozens of use cases, some sharing behaviour. UML provides relationships: **association** (an actor participates in a use case), **include** (one use case incorporates another as part of its execution — “this behaviour is always needed”; e.g. Register Course <<include>> Authenticate Student), **extend** (additional behaviour that occurs under particular circumstances — “sometimes this extra behaviour happens”; e.g. Withdraw Cash <<extend>> Report Suspicious Transaction), and **generalisation** (a specialised actor/use case inherits from a more general one — Student and Lecturer are specialised Users).

Memory trick: **INCLUDE** = “I need this as part of the job”; **EXTEND** = “I might add this under a condition”. Include factors common/reused behaviour; extend models optional/exceptional behaviour.

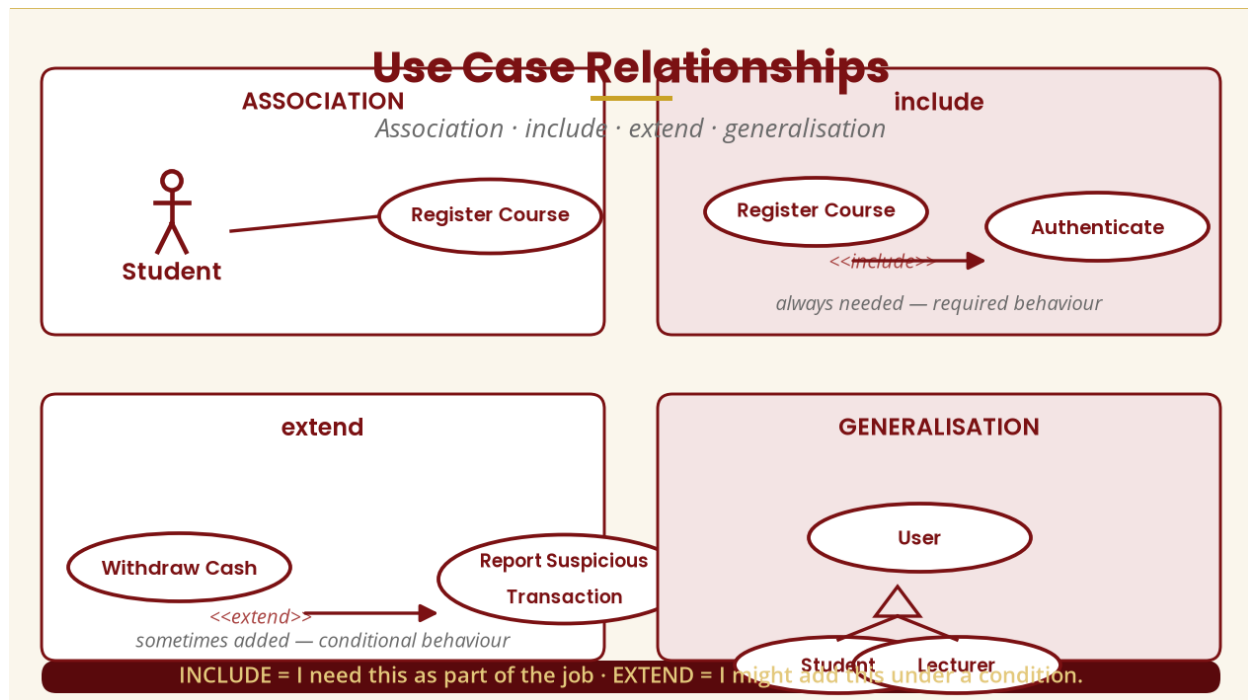


Figure 6 — The four use case relationships

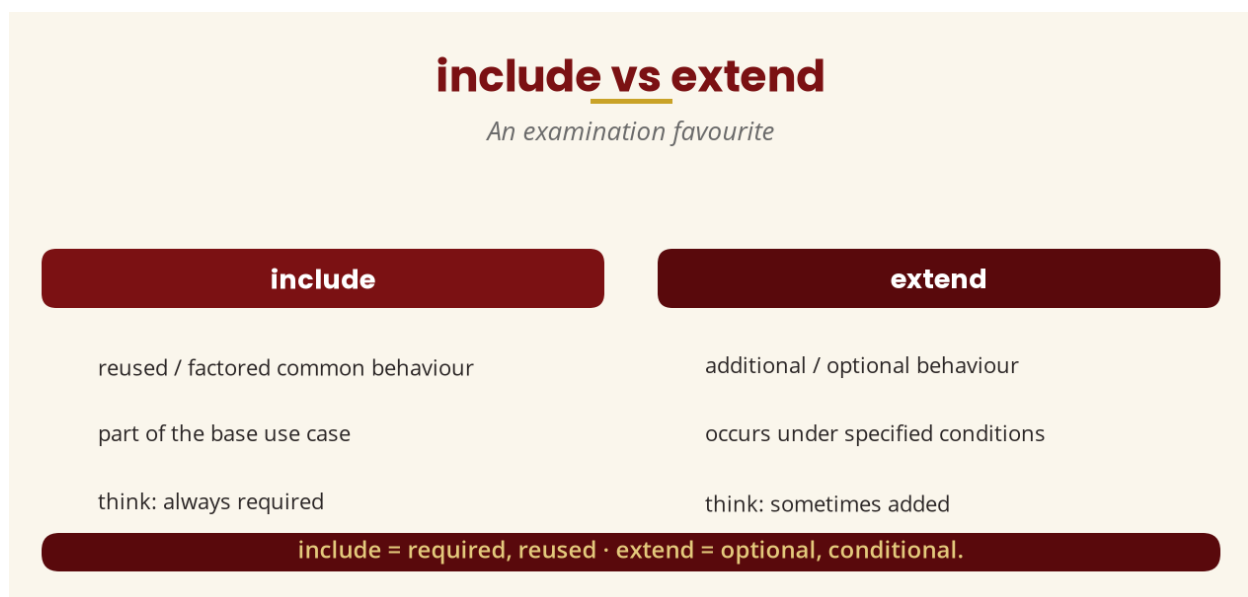


Figure 7 — include vs extend

PART E — USE CASE SPECIFICATIONS

8. The Diagram Is Not Enough

A use case diagram gives only a high-level view. Serious analysis needs a **use-case specification** describing what happens inside the use case — its ID, name, primary actor, goal, preconditions, main success scenario, alternative flows and postconditions. For Register for Course, the main scenario might be: select Register Course → view available courses → select a course → check prerequisites → check capacity → confirm → create registration → show confirmation. Alternative flows handle a full course (offer waitlist) and missing prerequisites (stop registration).

Telling a programmer only “build course registration” leaves them guessing what happens when a course is full or payment fails. The specification captures those requirements — which is why behavioural analysis connects requirements to design, and why we must model alternative and exception paths, not only the “happy path”.

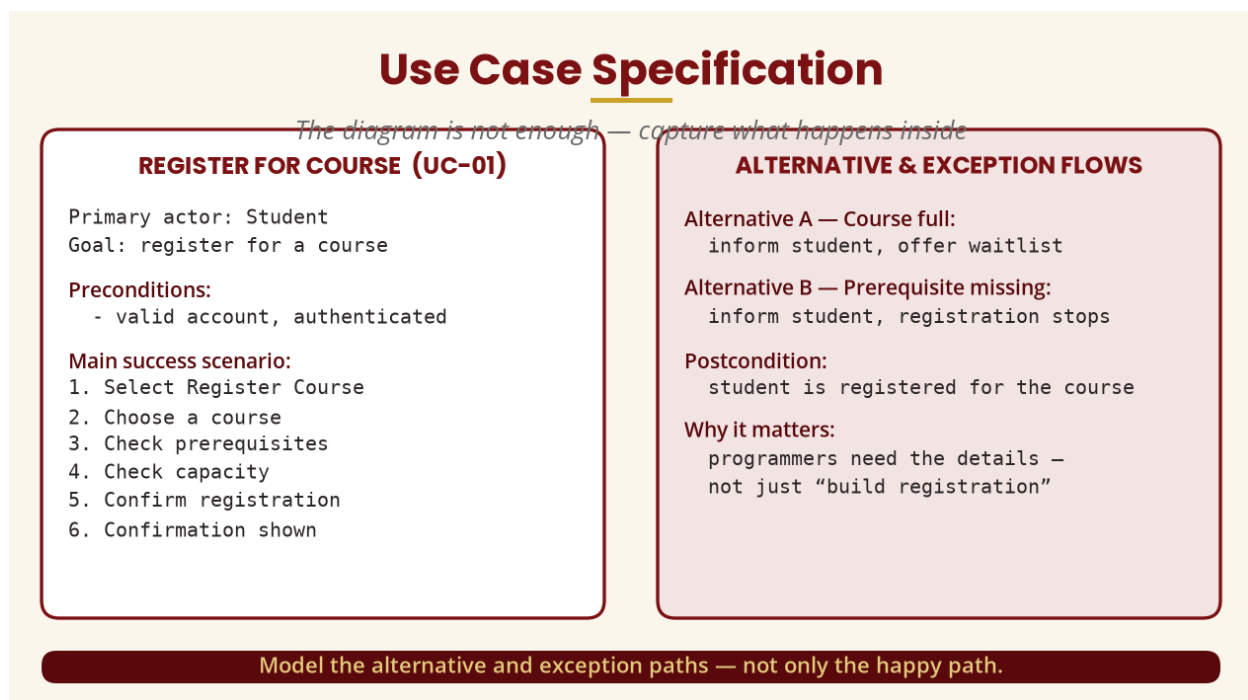


Figure 8 — A use case specification

PART F & G — INTERACTION AND SEQUENCE DIAGRAMS

9. Sequence Diagrams

An **interaction** describes how participants communicate by exchanging messages. A **sequence diagram** shows interactions in **time order** — who communicates with whom, what messages are exchanged, and in what order. Its components are: actor, object, **lifeline** (the vertical dashed line representing a participant's existence), **activation bar** (a period during which a participant is performing an operation), **message** (communication from one participant to another) and return message. Read from top to bottom: time flows downward.

A login sequence: Student → System login(); System → Database validate(); Database → System valid; System → Student confirmation. Order matters — you cannot send “login successful” before validating the credentials. A registration sequence introduces Registration UI, Registration Controller and Course Database, moving from requirements toward the software objects that will participate.

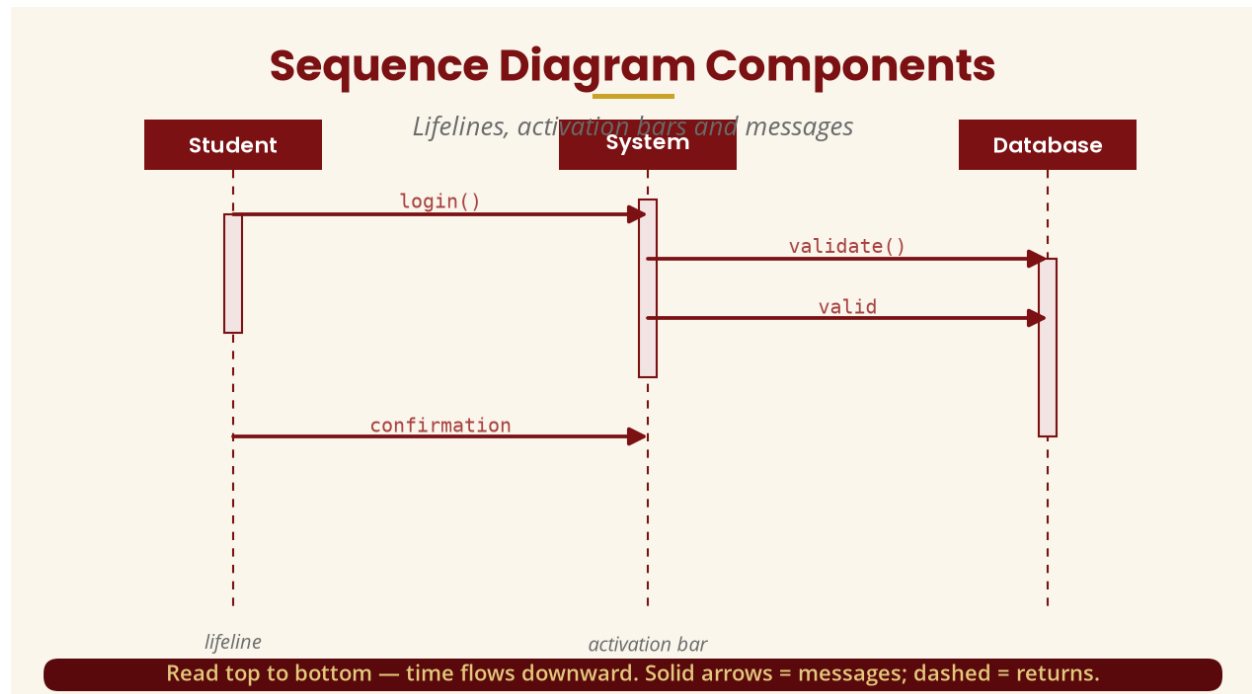


Figure 9 — Sequence diagram components

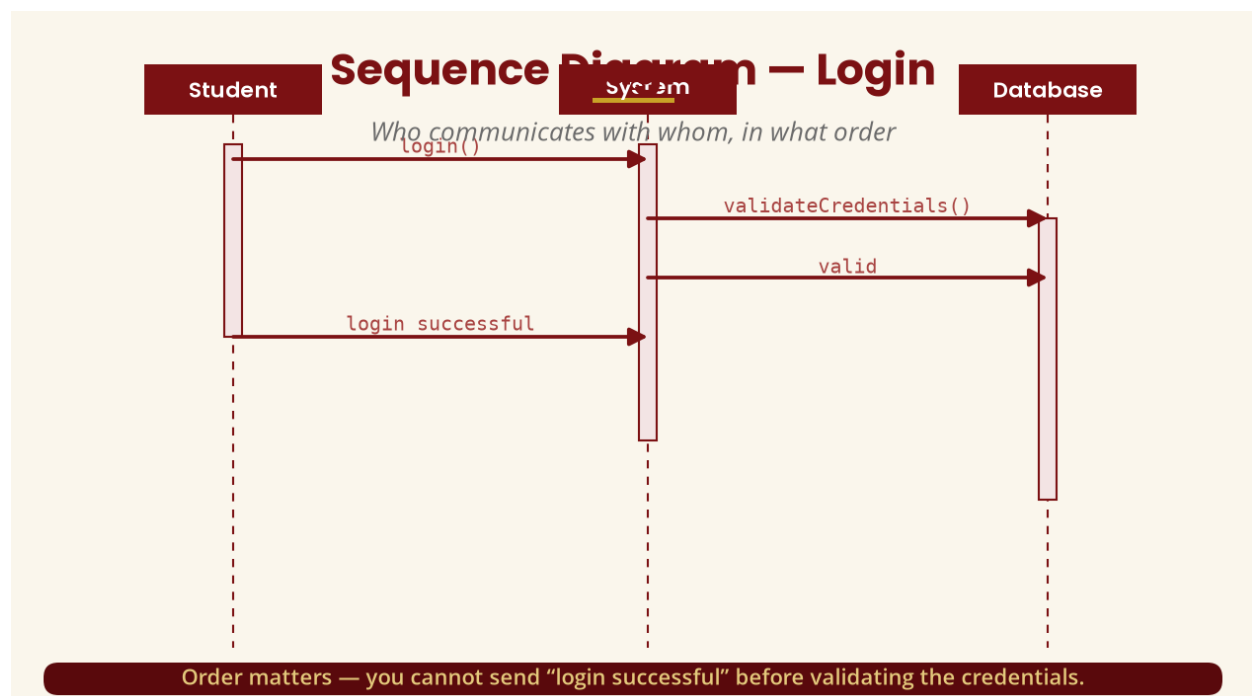


Figure 10 — A login sequence diagram

10. Alternative Sequences — alt, loop, opt

Real systems do not always follow the happy path. Combined fragments model alternatives: **alt** — only the branch whose guard is true executes (course available → register; course full → show message); **loop** — behaviour repeats (for each student, send the result); **opt** — behaviour occurs only if a condition is true (send SMS only if notifications are enabled).

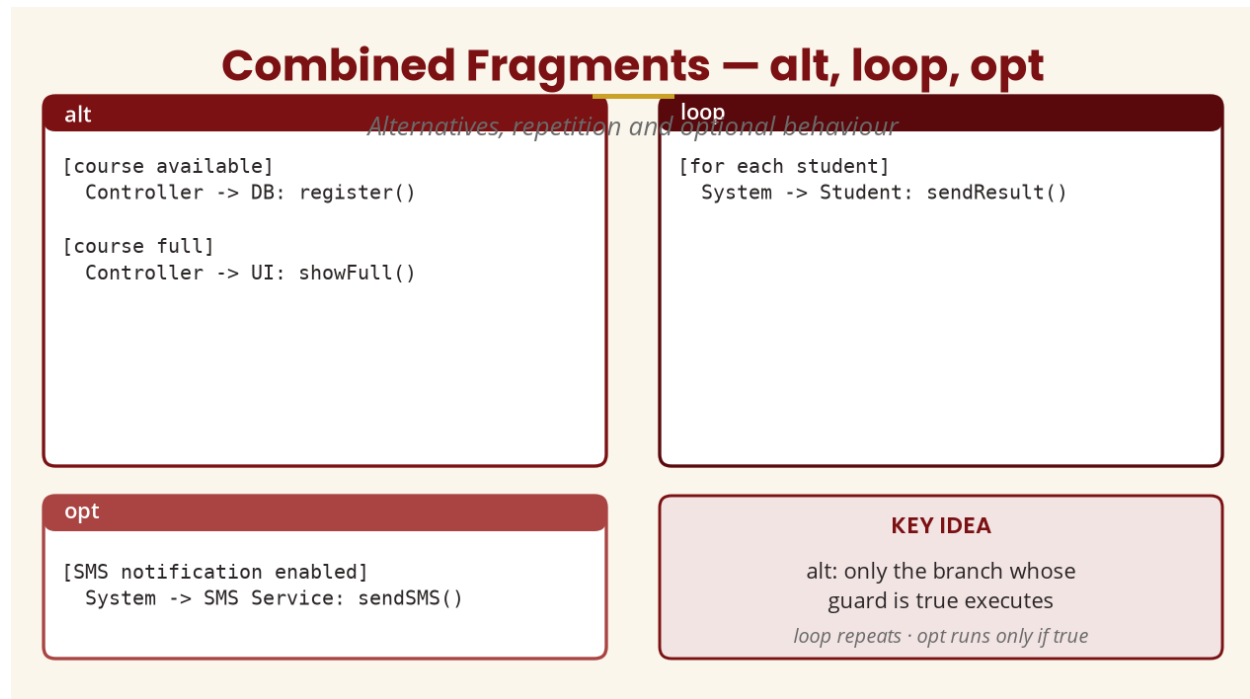


Figure 11 — Combined fragments: alt, loop, opt

PART I — COMMUNICATION (COLLABORATION) DIAGRAMS

11. Communication Diagrams

The course outline uses the older term **collaboration diagram**; in UML 2.x the standard term is **communication diagram** — students may meet both names. A communication diagram focuses on **which objects communicate and what messages they exchange**, emphasising object relationships rather than vertical time. Messages are numbered — 1: register(), 2: checkCourse(), 3: availability, 4: confirmation — to show order. The same interaction can be shown as a sequence diagram (emphasising time and message order) or a communication diagram (emphasising participating objects and their links).

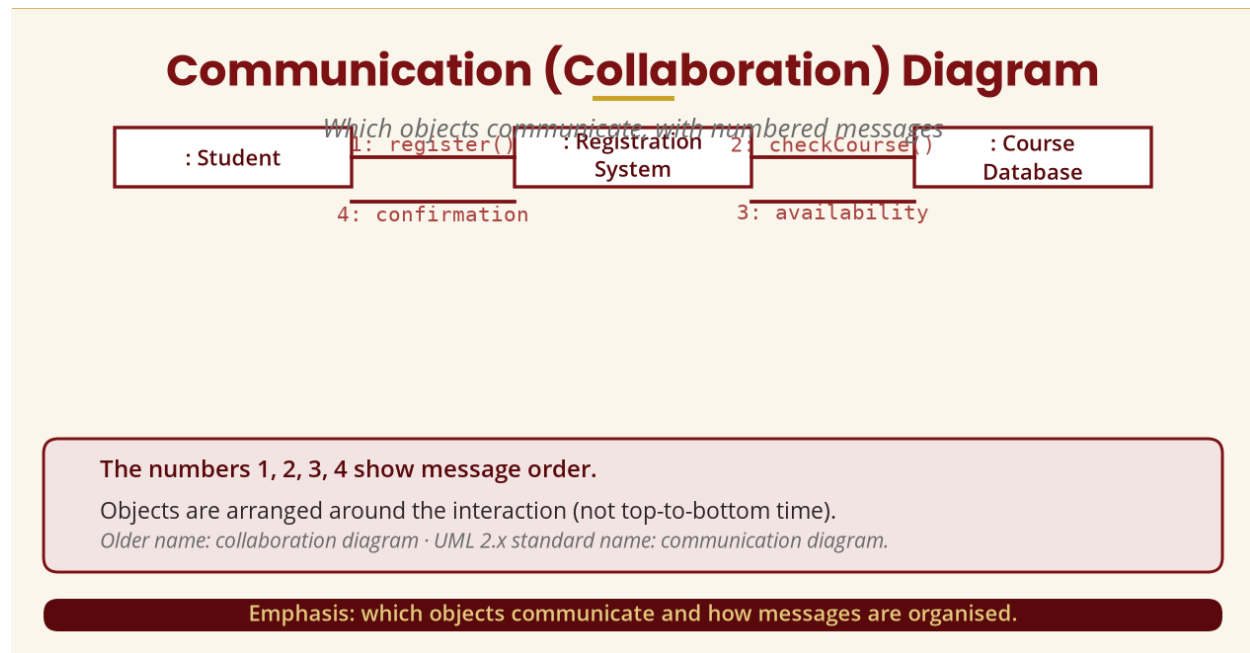


Figure 12 — A communication diagram

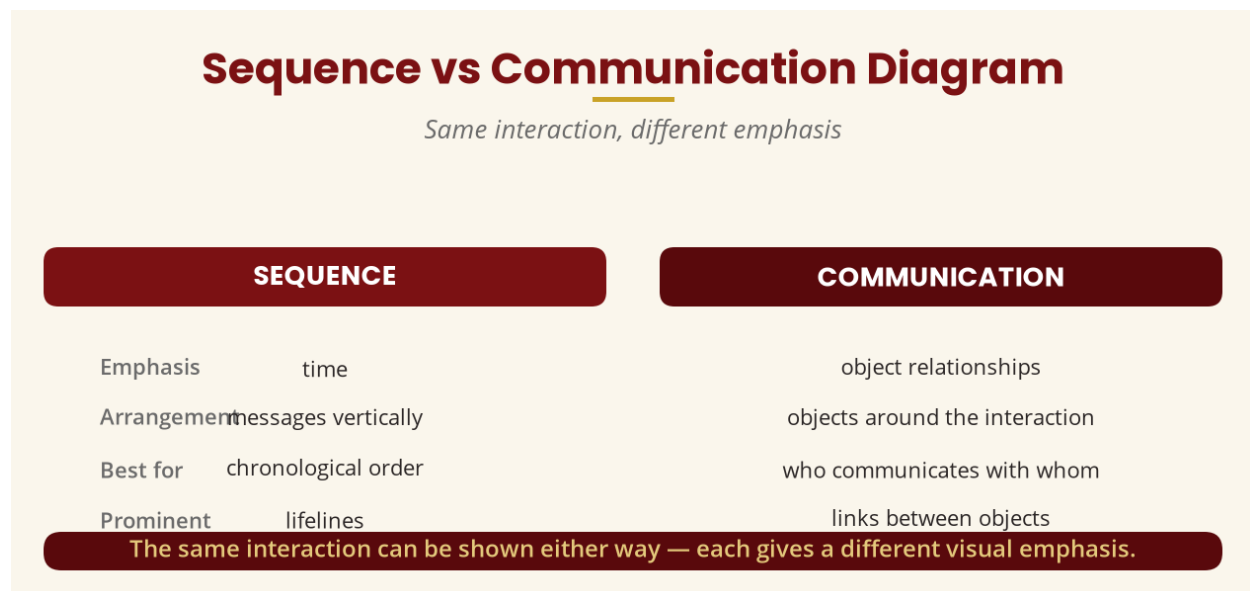


Figure 13 — Sequence vs communication diagram

PART J — ACTIVITY DIAGRAMS

12. Activity Diagrams

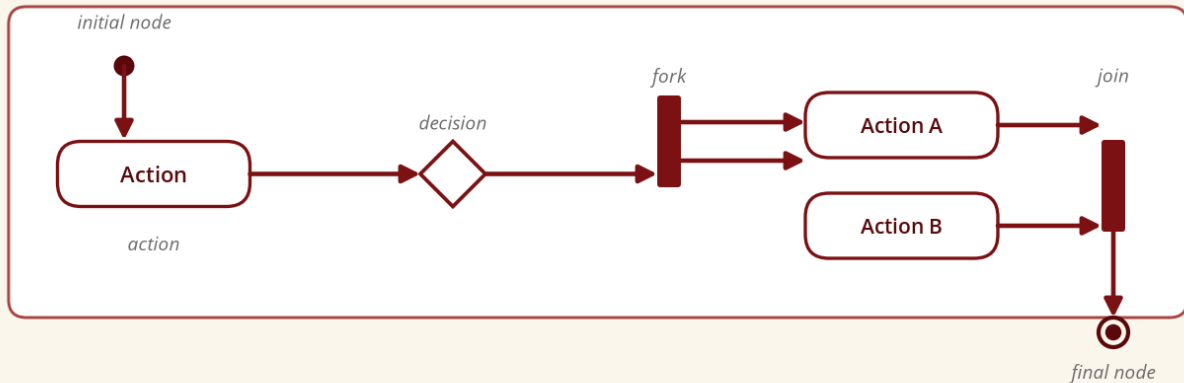
An **activity diagram** models a workflow or flow of activities — the steps involved in carrying out a process, including decisions and parallel activities. Its notation: the initial node (□), actions (rounded rectangles), the decision/merge diamond (◊), the final node (◻), flow arrows, and **fork/join** thick bars. The diamond represents a decision whose outgoing paths carry guard conditions — [balance sufficient] / [balance insufficient].

A fork splits one flow into multiple **concurrent** flows (after an order is confirmed: Prepare Food **and** Assign Driver, then join before Deliver Order). An online shop after payment may update

inventory, prepare an invoice, notify the warehouse and send confirmation — some of which can occur independently.

Activity Diagram Notation

Initial node · actions · decision · fork/join · final node

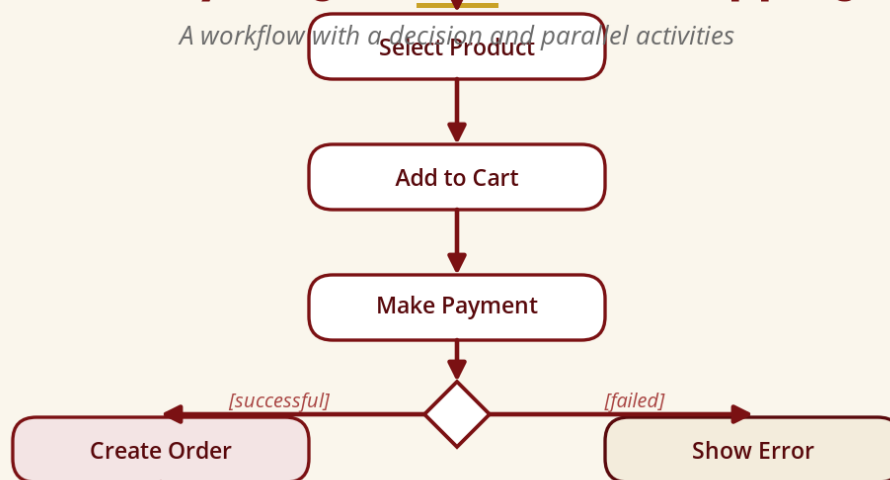


The diamond is a decision; the thick bars are fork (split) and join (synchronise).

Figure 14 — Activity diagram notation

Activity Diagram — Online Shopping

A workflow with a decision and parallel activities



After payment, the diamond branches: successful -> Create Order; failed -> Show Error.

Figure 15 — An online shopping activity diagram

13. Swimlanes

A **swimlane** divides an activity diagram according to responsibility, answering “**who is responsible for performing each activity?**” Without swimlanes, a list of activities (check prerequisites, confirm registration, update database) does not tell us who performs what; with swimlanes, Student performs Select course and Confirm, while System performs Check prerequisites, Create registration and Display confirmation.

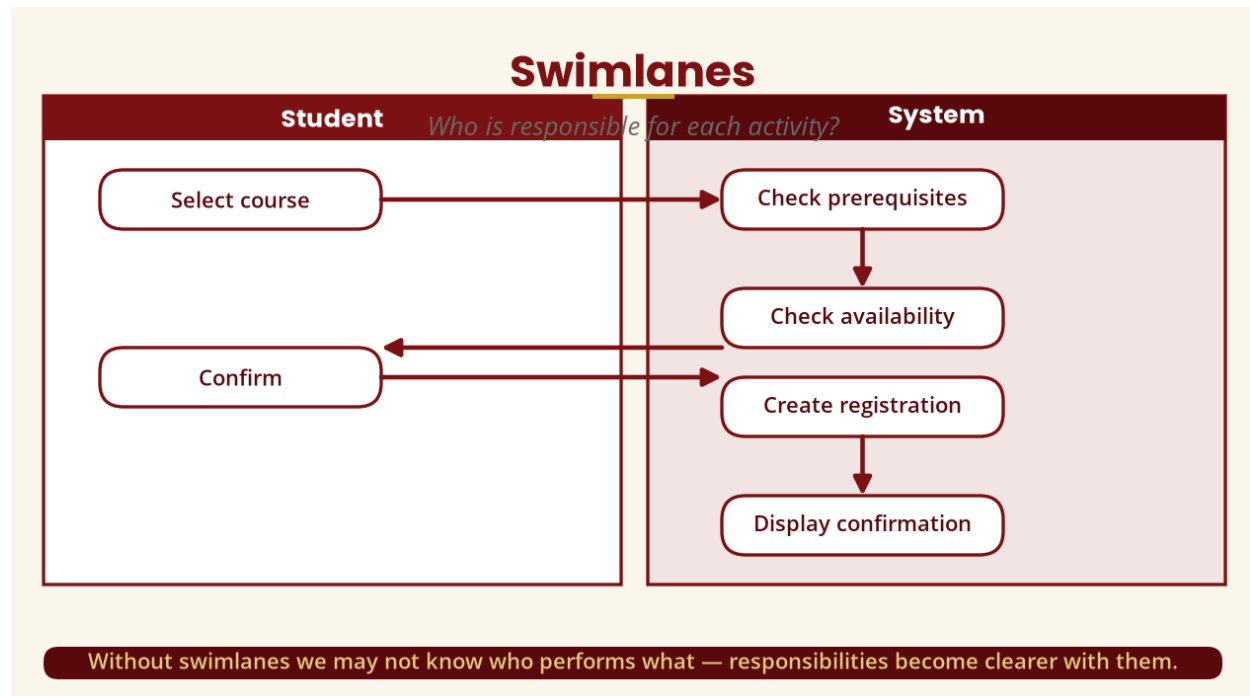


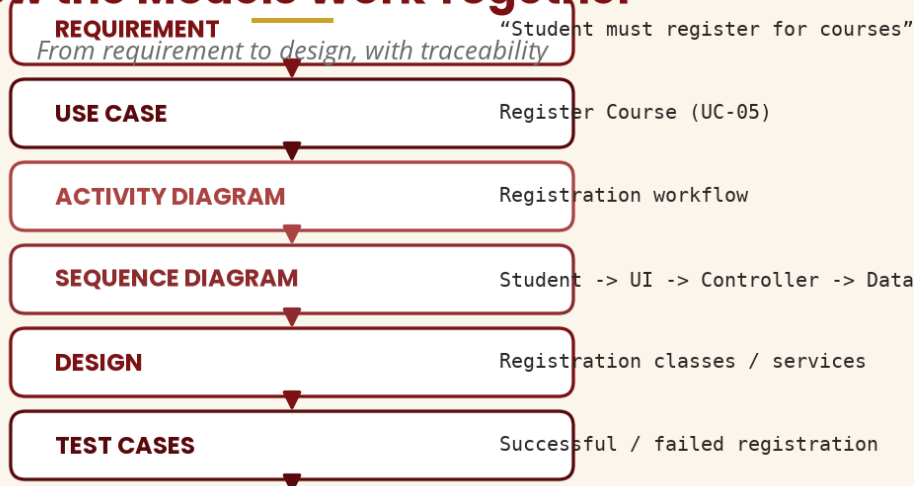
Figure 16 — Swimlanes

PART M — HOW THE MODELS WORK TOGETHER

14. Use Case Activity Sequence

The most important concept of Week 8: the models work together. From the requirement “a student should be able to register for a course”, we write the **use case** (Register Course), model the **workflow** with an activity diagram (select course → check prerequisite → check availability → confirm → create registration), then model the **object interactions** with a sequence diagram (Student → Registration UI → Registration Controller → Course Database). The chain runs: requirement → use case → activity diagram → sequence diagram → object responsibilities → design.

How the Models Work Together



Each model elaborates the same requirement at a deeper level — that is traceability.

Figure 17 — Requirement to design, with traceability

15. Full Analysis Documentation

Analysis documentation is the collection of models and written descriptions that explain what the proposed system must do: the problem statement, business/functional/non-functional requirements, actors, use cases, use-case and activity and sequence and communication diagrams, class and state-machine diagrams, assumptions, constraints, business rules and a glossary. It matters because the client, analyst, designer, programmer, tester and project manager all need a common language and **traceability** — the client says “register courses”, the analyst calls it UC-03, the designer draws the sequence diagram, the programmer identifies the objects, and the tester derives the scenarios. REQ-05 □ UC-05 □ activity model □ sequence model □ design □ test cases.

16. Common Student Confusions

Confusion	Clarification
Use case vs use case diagram	A use case is one goal/behaviour (Register Course); the diagram is the visual representation of actors, use cases and relationships.
Use case vs activity diagram	Use case = what goal; activity diagram = what workflow happens.
Activity vs sequence	Activity = workflow/process (check stock → process payment); sequence = messages between participants over time.
Sequence vs communication	Sequence emphasises time/order; communication emphasises objects and links.
Actor vs class	An actor is external to the system (Student); a class is a type inside the model (StudentAccount).

17. Practical Lab and Tutorial

Practical — University Online Course Registration System: identify at least four actors (Student, Lecturer, Administrator, Payment Gateway) and eight use cases (Login, Register Course, Drop Course, View Timetable, Make Payment, View Results, Enter Marks, Generate Report); draw a complete use-case diagram with a system boundary, associations, at least one include, one extend and one generalisation; write a full specification for Register Course; draw an activity diagram with decisions, alternative paths and swimlanes; draw a sequence diagram (Student, Registration Interface, Registration Controller, Course Database) with at least eight messages; and represent the same scenario as a numbered communication diagram.

Tutorial: (1) for an ATM system identify at least three actors, six use cases and one external system; (2) explain include vs extend vs generalisation with practical examples; (3) decide whether an activity diagram or a sequence diagram best fits a given workflow or message exchange; (4) explain why a sequence diagram should model a specific scenario rather than an entire system in one enormous diagram.

18. Examination–Style Questions

Question 1 (Definitions). Define behavioural modelling, use case, actor, interaction diagram, sequence diagram, communication diagram and activity diagram. [14 marks]

Question 2 (Comparison). Differentiate between: use case and use case diagram; activity diagram and sequence diagram; sequence diagram and communication diagram; include and extend. [20 marks]

Question 3 (Design). A university wants an online examination system where students log in, view examinations, start an examination, answer questions, submit answers and receive confirmation, and lecturers create examinations, enter questions, view submissions and generate results. Identify the actors and use cases, draw a use-case diagram, write a use-case

specification for Start Examination, draw an activity diagram for Submit Examination, and draw a sequence diagram for Start Examination.

19. Week 8 Summary and Key Terms

Concept	Meaning
Behavioural modelling	Models system behaviour — the registration process.
Use case / actor	A user/system goal · an external role.
System boundary	The rectangle defining system scope.
include / extend	Required/reused behaviour · conditional additional behaviour.
Sequence diagram	Messages in time order (lifelines, activation bars).
Communication diagram	Objects and numbered messages (collaboration).
Activity diagram	A workflow with decisions, forks/joins and swimlanes.
Use-case specification	The detailed textual behaviour inside a use case.
Full analysis documentation	All models + descriptions, with traceability.

THE BIG PICTURE

A system is not just a collection of classes — it is a collection of behaviours performed by users and software objects working together to achieve goals. From “I need a registration system”: **Who?** Student □ **What goal?** Register Course □ **What workflow?** activity diagram □ **Which objects interact?** sequence diagram □ **How are they connected?** communication diagram □ **What does the complete requirement say?** full analysis documentation. The OMG UML 2.5.1 specification remains the key normative reference for these constructs.

20. References and Further Reading

Primary authoritative reference

- Object Management Group. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. OMG — the formal specification, including use cases, sequence diagrams, communication diagrams and activity diagrams.
- Object Management Group. *UML* — official UML resources, specifications and guidance.

Additional behavioural modelling references

- Microsoft. *UML diagrams in Visio* — overview of sequence, activity, state-machine, use-case and communication diagrams.

-
- NILUS Consulting. (2026). *Object-Oriented Analysis and Design Using UML — Behavioral Modeling with Sequence, Activity, and State Diagrams*.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling*.
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML*.
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide*.
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design*.
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements*.
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML*.